

Inheritance

林致宇

Review

- Object Oriented
 - Encapsulation / Abstraction / Inheritance / Polymorphism
- 繼承是重複使用並擴展現有類別定義以產生新類別的方法
- 繼承也是 Polymorphism 的基礎

Outline

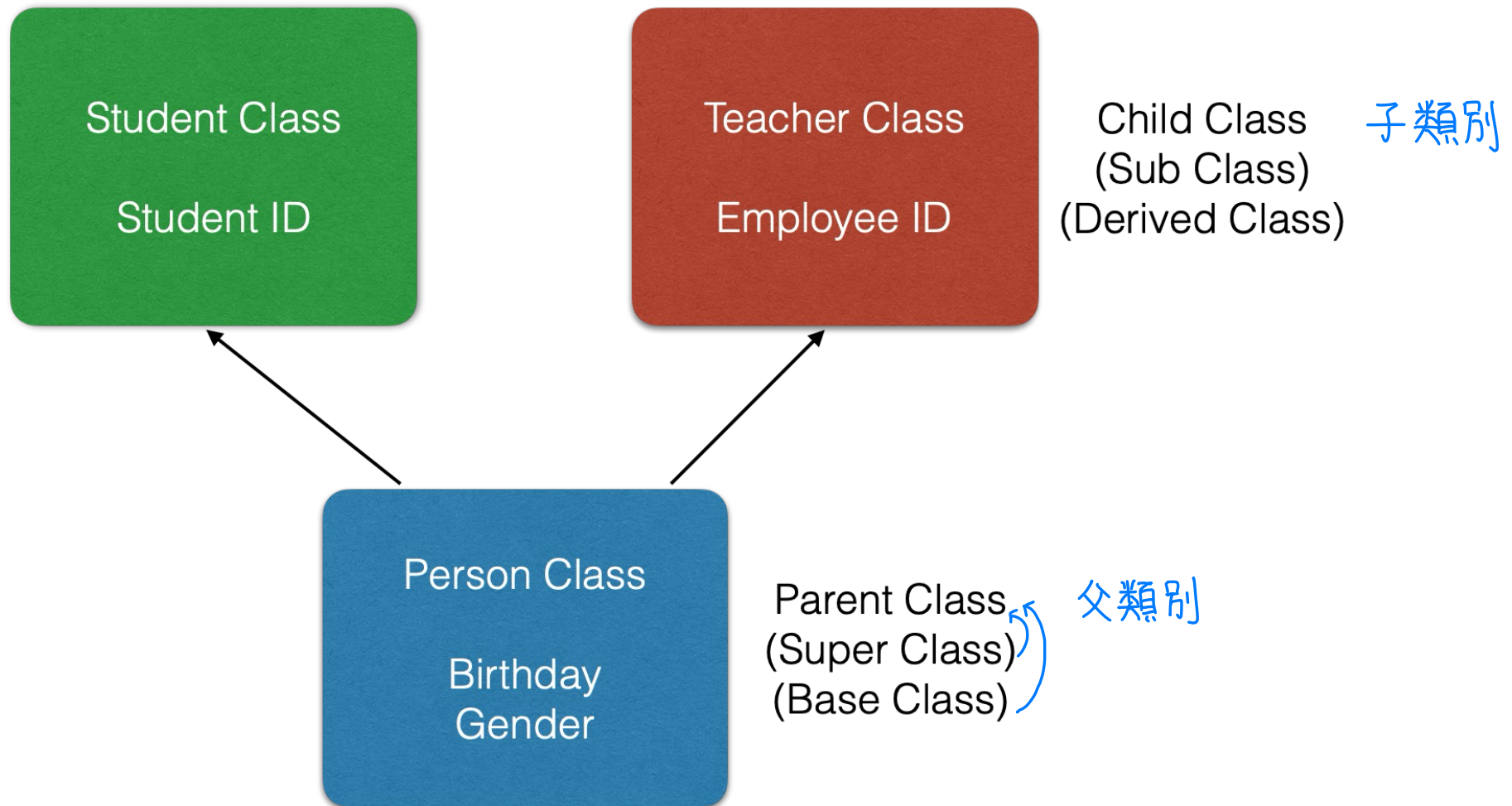
- Inheritance
- Access Specifier
- Constructor
- Member Variables
- Member Functions
- Multiple Inheritance

Outline

- Inheritance
- Access Specifier
- Constructor
- Member Variables
- Member Functions
- Multiple Inheritance

Inheritance

The Student class is derived from the Person class.



Inheritance

```
class <derived_class_name> : <access-specifier> <base_class_name> {  
    //body  
};
```

```
4 class Shape {  
5 public:  
6     double area;  
7     void draw() {  
8         cout << "draw() is called." << endl;  
9     }  
10 };  
11  
12 class Circle: public Shape {  
13 public:  
14     double radius;  
15 };
```

circle 繼承了 Shape

也可以繼承 private (不考)

```
17 int main(int argc, const char * argv[]) {  
18     Circle c;  
19     c.radius = 2.0;  
20     c.area = c.radius*c.radius*3.14;  
21     c.draw();  
22 }
```

Protected Member Variables

```
4 class Shape {
5 private:
6     int pri_member;
7 protected:
8     int pro_member;
9 public:
10 };
11
12 class Circle: public Shape {
13     void accessProMember() { pro_member = 6; }
14     // error void accessPriMember() { pri_member = 5; } // error
15 };
16
17 int main(int argc, const char * argv[]) {
18     Circle c;
19     // c.pri_member = 5; // error
20     // c.pro_member = 6; // error
21 }
```

■ protected 是專門為繼承而設的
■ protected 是專供家族子孫所私用的物品

protected 類似 private，只是可以被繼承！

Circle 不存在 pri_member
(不能繼承 private)

	Public	Protected	Private
類別外可用	✓	✗	✗
可被繼承	✓	✓	✗

Outline

- Inheritance
- Access Specifier
- Constructor
- Member Variables
- Member Functions
- Multiple Inheritance

Access Specifier ~~不考~~

Access specifier in base class	Access specifier when inherited publicly
Public	Public
Protected	Protected
Private	Inaccessible

Access Specifier ✗不考

```
4  class Shape {
5  private:
6      int pri_member;
7  protected:
8      int pro_member;
9  public:
10     int pub_member;
11 };
12
13 class Circle: public Shape {
14     void accessProMember() { pro_member = 6; }
15 };
16
17 int main(int argc, const char * argv[]) {
18     Circle c;
19     c.pub_member = 5;
20 }
```

Access Specifier X 不考

Access specifier in base class	Access specifier when inherited protectedly
Public	Protected
Protected	Protected
Private	Inaccessible

Access Specifier ✗不考

```
4  class Shape {
5  private:
6      int pri_member;
7  protected:
8      int pro_member;
9  public:
10     int pub_member;
11 };
12
13 class Circle: protected Shape {
14     void accessProMember() { pro_member = 6; }
15 };
16
17 int main(int argc, const char * argv[]) {
18     Circle c;
19     c.pub_member = 5; // error
20 }
```

Access Specifier ~~不考~~

Access specifier in base class	Access specifier when inherited privately (Default)
Public	Private
Protected	Private
Private	Inaccessible

Access Specifier ✗不考

```
4  class Shape {
5  private:
6      int pri_member;
7  protected:
8      int pro_member;
9  public:
10     int pub_member;
11 };
12
13 class Circle: private Shape {
14     void accessProMember() { pro_member = 6; }
15 };
16
17 int main(int argc, const char * argv[]) {
18     Circle c;
19     // c.pub_member = 5; // error
20 }
```

Inheritance

不考，但要看一下

- In principle, a publicly derived class inherits access to every member of a base class except:
 - its constructors and its destructor 建構子無法被繼承
 - its assignment operator members (operator=)
 - its friends
 - its private members

Outline

- Inheritance
- Access Specifier
- Constructor
- Member Variables
- Member Functions
- Multiple Inheritance

Constructor

但Constructor是不能被繼承的!
Circle 中沒有一個Constructor叫 Shape。
(但建Circle時 Shape會先被呼叫)

```
4  class Shape {  
5  public:  
6      double area;  
7      Shape() { cout << "Shape Constructor" << endl; }  
8  };  
9  
10 class Circle: public Shape {  
11 public:  
12     double radius;  
13     Circle() { cout << "Circle Constructor" << endl; }  
14 };  
15  
16 int main(int argc, const char * argv[]) {  
17     Circle c;  
18 }
```

父類別、子類別都有Constructor時
當Create子類別時，父類別的Constructor
會先被呼叫，在呼叫子類別的Constructor。

```
[lincyu@Chih-YudeMacBook-Pro Chapter12 % ./a.out  
Shape Constructor  
Circle Constructor
```

Constructor

```
lincyu@Chih-YudeMacBook-Pro Chapter12 % g++ 12_06_Constructor2.cpp
12_06_Constructor2.cpp:13:5: error: constructor for 'Circle' must explicitly initialize the base class 'Shape' which does not have a default constructor
```

```
Circle() { cout << "Circle Constructor" << endl; }
```

^

```
4 class Shape {
5 public:
6     double area;
7     Shape(int x) { cout << x << endl; }
8 };
9
```

```
10 class Circle: public Shape {
11 public:
12     double radius;
13     Circle() { cout << "Circle Constructor" << endl; }
14 };
15
```

```
16 int main(int argc, const char * argv[]) {
17     Circle c; → CE!
18 }
```

💡 定義有參數的 Constructor 時，
default 的無參數 Constructor 失效！

Constructor

```
4  class Shape {
5  public:
6      double area;
7      Shape(int x) { cout << x << endl; }
8  };
9
10 class Circle: public Shape {
11 public:
12     double radius;
13     Circle(): Shape(5) { cout << "Circle Constructor" << endl; }
14     Circle(int x, double r): Shape(x), radius(r) {}
15 };
16
17 int main(int argc, const char * argv[]) {
18     Circle c1;
19     Circle c2(6, 3.0);
20     Circle c3(8); // error
21 }
```

因底 Shape Constructor 必須要有參數！

Default Copy Constructor

```
4  class Shape {
5  public:
6      int x;
7  };
8
9  class Circle: public Shape {
10 public:
11     int y;
12 };
13
14 int main(int argc, const char * argv[]) {
15     Circle c1;
16     c1.x = 5;
17     c1.y = 3;
18     Circle c2 = c1;
19     cout << c2.x << ", " << c2.y << endl;
20 }
```

Default Copy Constructor

```
4  class Shape {
5  public:
6      int x;
7      Shape() {} ← Constructor
8      Shape(const Shape &s);
9  };
10
11  Shape::Shape(const Shape &s) {
12      this->x = s.x + 2;
13  }
14
15  class Circle: public Shape {
16  public:
17      int y;
18  };
```

只有在 Line 8 的父類別中有 Copy Constructor.

If the parent class has defined its copy constructor, the copy constructor will be called.

```
20  int main(int argc, const char * argv[]) {
21      Circle c1; → Call Line 7
22      c1.x = 5;
23      c1.y = 3;
24      Circle c2 = c1; → Call Line 8
25      cout << c2.x << ", " << c2.y << endl;
26  }
```

Default Copy Constructor

```
4  class Shape {
5  public:
6      int x;
7      Shape() {}
8      Shape(const Shape &s);
9  };
10
11 Shape::Shape(const Shape &s) {
12     this->x = s.x + 2;
13 }
14
15 class Circle: public Shape {
16 public:
17     Circle() {}
18     Circle(const Circle &c);
19     int y;
20 };
21
22 Circle::Circle(const Circle &c) {
23     this->y = c.y + 3;
24 }
```

If the child class and parent class have defined their copy constructor, the copy constructor of the parent class will not be called!

Line 8 的父類別、Line 18 的子類別都有 Copy Constructor.

```
lincyu@Chih-YudeMacBook-Pro Chapter12 % g++ 12_08_CopyConstructor3.cpp
lincyu@Chih-YudeMacBook-Pro Chapter12 % ./a.out
0, 6
```

```
26 int main(int argc, const char * argv[]) {
27     Circle c1;
28     c1.x = 5;
29     c1.y = 3;
30     Circle c2 = c1;
31     cout << c2.x << ", " << c2.y << endl;
32 }
```

只有子類別的 Copy Constructor 會被 call
父類別的 Copy Constructor 不會被 call

Default Copy Constructor

```
22  Circle::Circle(const Circle &c): Shape(c) {  
23      this->y = c.y + 3;  
24  }  
25  
26  int main(int argc, const char * argv[]) {  
27      Circle c1;  
28      c1.x = 5;  
29      c1.y = 3;  
30      Circle c2 = c1;  
31      cout << c2.x << ", " << c2.y << endl;  
32  }
```

Outline

- Inheritance
- Access Specifier
- Constructor
- Member Variables
- Member Functions
- Multiple Inheritance

Shadow

```
4  class Shape {
5  public:
6      int x;
7  };
8
9  class Circle: public Shape {
10 public:
11     double x; ← 遮蔽效应, 父的x会被遮蔽
12     void showX() {
13         cout << x << ", " << Shape::x << endl;
14     }
15 };
16
17 int main(int argc, const char * argv[]) {
18     Circle c;
19     c.x = 5;
20     c.Shape::x = 7; ← 若真的要存取父類別的x, 要用
21     c.showX();      範圍解析運算子。
22 }
```

Outline

- Inheritance
- Access Specifier
- Constructor
- Member Variables
- Member Functions
- Multiple Inheritance

Override

```
4  class Shape {
5  public:
6      double area;
7      void draw() { cout << "1: draw()" << endl; }
8  };
9
10 class Circle: public Shape {
11 public:
12     double radius;
13     void draw() { cout << "2: draw()" << endl; }
14 };
15
16 int main(int argc, const char * argv[]) {
17     Circle c;
18     c.draw(); ←子類別的 Draw 才會被呼叫
19 }
```

Circle::draw 覆寫 (Override) 了父類別的 Shape::draw

和 Constructor 很不一樣!

Override

```
10  class Circle: public Shape {
11  public:
12      double radius;
13      void draw() {
14          Shape::draw(); // No super in C++
15          cout << "2: draw()" << endl;
16      }
17  };
```

↓ 用範圍解析運算子, 呼叫父類別的 draw

Outline

- Inheritance
- Access Specifier
- Constructor
- Member Variables
- Member Functions
- Multiple Inheritance Java 也沒有多重繼承

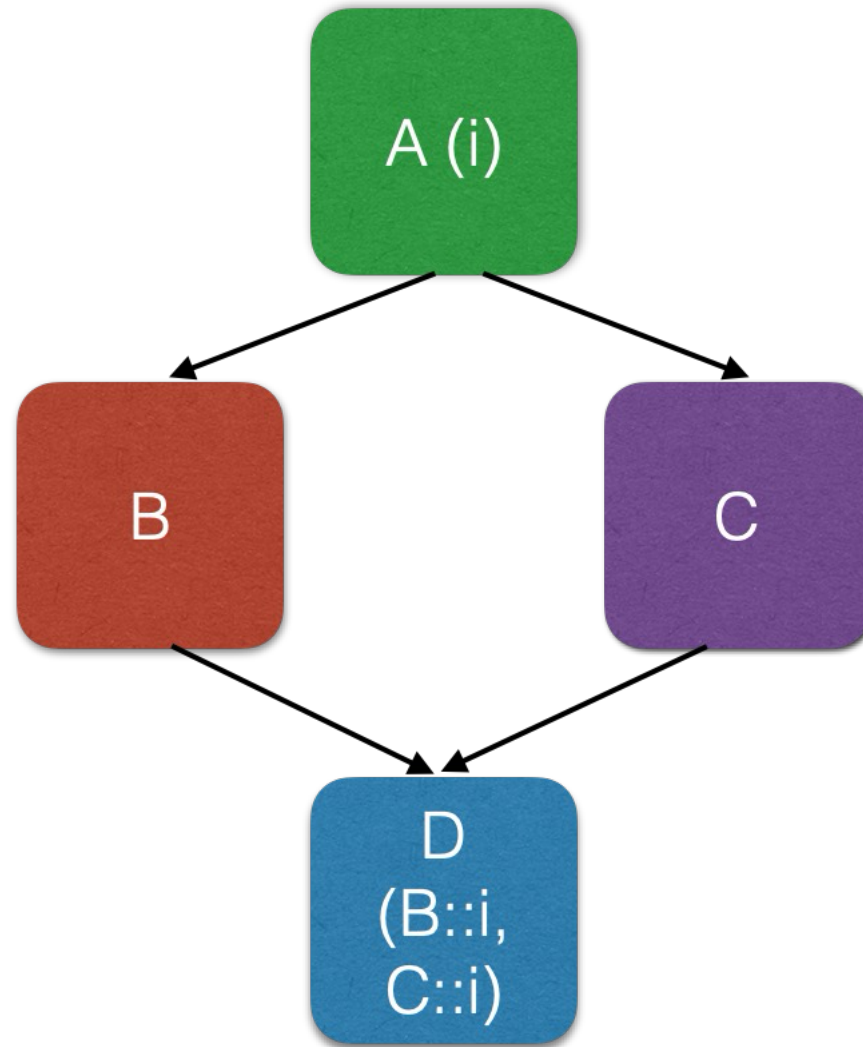
Multiple Inheritance

```
4  class Vehicle {
5  public:
6      Vehicle() { cout << "Vehicle" << endl; }
7  };
8
9  class ExerciseTool {
10 public:
11     ExerciseTool() { cout << "ExerciseTool" << endl; }
12 };
13
14 class Bicycle: public ExerciseTool, public Vehicle {
15 public:
16     Bicycle() { cout << "Bicycle" << endl; }
17 };
18
19 int main(int argc, const char * argv[]) {
20     Bicycle bike;
21 }
```

Members that have the same name

```
4  class Vehicle {
5  ~ public:
6      double speed;
7      double getSpeed() { return speed; }
8  };
9
10 class ExerciseTool {
11 ~ public:
12     double speed;
13     double getSpeed() { return speed; }
14 };
15
16 class Bicycle: public ExerciseTool, public Vehicle {
17 };
18
19 ~ int main(int argc, const char * argv[]) {
20     Bicycle bike;
21     bike.Vehicle::speed = 3.0;
22     bike.ExerciseTool::speed = 5.0;
23     cout << bike.Vehicle::getSpeed() << endl;
24     cout << bike.ExerciseTool::getSpeed() << endl;
25 }
```

The same base class



The same base class

```
4  class A {
5  public:
6      int x;
7  };
8  class B: public A {
9  };
10 class C: public A {
11 };
12 class D: public B, public C {
13 };
14
15 int main() {
16     D d;
17     d.B::x = 3;
18     d.C::x = 5;
19     cout << d.B::x << ", " << d.C::x << endl;
20     // cout << d.A::x << endl; // error
21 }
```

```
lincyu@Chih-YudeMacBook-Pro Chapter12 % g++ 12_13_SameBaseClass.cpp
12_13_SameBaseClass.cpp:20:13: error: ambiguous conversion from derived cl
ass 'D' to base class 'A':
    class D -> class B -> class A
    class D -> class C -> class A
    cout << d.A::x << endl;
           ^
1 error generated.
lincyu@Chih-YudeMacBook-Pro Chapter12 % g++ 12_13_SameBaseClass.cpp
lincyu@Chih-YudeMacBook-Pro Chapter12 % ./a.out
3, 5
lincyu@Chih-YudeMacBook-Pro Chapter12 %
```

Virtual base class

```
4  class A {
5  public:
6      int x;
7  };
8  class B: public virtual A {
9  };
10 class C: public virtual A {
11 };
12 class D: public B, public C {
13 };
14
15 int main() {
16     D d;
17     d.B::x = 3;
18     d.C::x = 5;
19     cout << d.B::x << ", " << d.C::x << endl;
20 }
```

Reference:

https://en.wikipedia.org/wiki/Virtual_inheritance

Partial Virtual Inheritance

```
4  class A {
5  public:
6      int x;
7  };
8  class B1: public virtual A {};
9  class B2: public virtual A {};
10 class B3: public A {};
11 class C: public B1, public B2, public B3 {};
12
13 int main() {
14     C c;
15     c.B1::x = 1;
16     c.B2::x = 2;
17     c.B3::x = 3;
18     cout << c.B1::x << ", "
19         << c.B2::x << ", "
20         << c.B3::x << endl;
21 }
```

Different Access Specifier

```
4  class A {  
5  public:  
6      int x;  
7  };  
8  class B1: private virtual A {};  
9  class B2: public virtual A {};  
10 class C: public B1, B2 {};  
11  
12 int main() {  
13     C c;  
14     c.x = 3; Compiler Error!  
15 }
```

```
4  class A {  
5  public:  
6      int x;  
7  };  
8  class B1: private virtual A {};  
9  class B2: public virtual A {};  
10 class C: public B2, B1 {};  
11  
12 int main() {  
13     C c;  
14     c.x = 3; OK!  
15 }
```

Partial Overwrite

```
4  class A {
5  public:
6      void f() { cout << "A" << endl; }
7  };
8  class B1: public virtual A {
9  public:
10     void f() { cout << "B1" << endl; }
11 };
12 class B2: public virtual A {};
13 class C: public B1, B2 {};
14
15 int main() {
16     C c;
17     c.f();
18     c.A::f();
19 }
```